Data Structures and Algorithms

Lab 3 – Version 4

Stacks, Queues, and Linked Lists

Fundamentals, Real-World Applications, and Interview Practice

Lab Overview

This lab introduces three fundamental abstract data types and their implementations: **stacks** for last-in-first-out work, **queues** for first-in-first-out work, and **linked lists** for sequences connected by references. You will compare ordinary and specialized variants, trace state changes, read complete Java examples, and choose a representation based on the operations a problem requires.

The lab develops skills in implementing data structures, preserving invariants, working with empty and one-element structures, analyzing time and auxiliary space, and using structures to solve realistic problems. This is a **one-week individual Java assignment**. Each required program must be tested and explained.

Assignment Scope

Component	What to complete
Core problems	Problems **1, 3, 4, 5, 7, 9, and 10**.
Applied choice	Choose **one** of Problems **2 or 8**; submit Java code and its report section.
Interview choice	Choose **one** of Problems **6 or 11**; submit Java code and its report section.
Written practice	Interview Corner A–D, comparison table, technique-recognition exercise, and reflection: **PDF report only**.
Optional extension	Problem **12** and any unselected choice problems.

Complete **nine required programming problems**. Some problems compare two representations or algorithms. Each exercise includes a complete input/output contract, implementation guidance, and a worked example; sample input is read exactly as shown. The separate Interview Corner needs written reasoning, traces, or short snippets, **not additional runnable programs**.

Three guiding questions

Which end is accessible? A stack uses the top; a queue uses the front and back.
How is the sequence stored? An array has indexed positions; a linked list follows references.
Which operations matter? Choose the representation and variant from the workload, not from its name.

Programming and Analysis Requirements

Programming Rules

- Use **Java**. Implement your own array stack, circular-array queue, singly linked list, doubly linked list, and linked queue where explicitly required.
- Java `ArrayDeque` and `PriorityQueue` are allowed when a problem explicitly names them; do not replace a required manual implementation with a library container.
- Put algorithms into named methods or small classes. Include a driver or test harness for each submitted problem. A graphical user interface and interactive menu are not required.
- Represent links using references to `Node` objects, not arrays of simulated links. Do not use `java.util.LinkedList` where custom links are required.
- The input contract is part of the problem. If the contract allows an empty structure, handle it; do not silently return `0` as an empty-stack value when zero is valid data.

Common Conventions

Commands and examples operate in the listed order. Unless a problem states otherwise, identifiers and values are signed Java `int`s, input is well-formed, and the sequence has at most 10^5 operations. A failed pop, dequeue, or peek should use the problem's specified `EMPTY` result (or an explicitly documented exception in reusable APIs). For capacity-limited queues, $K \geq 1$. Preserve the FIFO or LIFO ordering exactly.

Analysis and Testing

For each required program, record the idea, an invariant or short correctness argument, worst-case time and auxiliary space (and amortized/expected bounds when appropriate), answers to its questions, and at least **three tests**: normal, boundary, and tricky. Show input, expected output, and actual output. Distinguish the space used to store n data items from extra working space used by an algorithm acting on an existing collection.

Review 1 – Abstract Data Types and Representation

Behavior versus Implementation

An **abstract data type (ADT)** defines permitted operations and their observable behavior; it does not require a particular storage representation. The same stack ADT can be implemented with an array or a linked list. Conversely, a linked list can implement multiple ADTs, including a stack and a queue.

ADT	Observable rule	Illustrative application
Stack	Last inserted item is removed first (LIFO).	Undo history, nested delimiters, depth-first work.
Queue	First inserted item is removed first (FIFO).	Customer tickets, breadth-first work, print requests.
Deque	Both ends support insertion and removal.	Work scheduling, rolling extrema.
Priority queue	Removal follows priority, not arrival alone.	Dispatch highest-urgency work.
List	Items have an explicit sequence and editable links.	Playlists, editable ordered records.

Array Versus Node References

An array provides $O(1)$ access by index. A singly linked list stores a value and reference to its next node; finding the ith node takes $O(i)$ traversal from the head. Having a sorted linked list does *not* make ordinary midpoint binary search an $O(\log n)$ array-like operation, because reaching the midpoint still takes traversal.

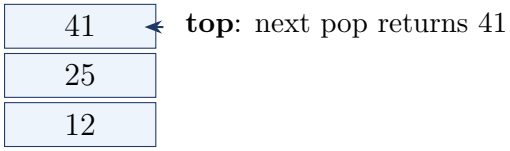
Complexity Refresher

$O(1)$ means bounded work per operation, $O(n)$ means proportional to the stored items, and $O(\log n)$ appears when a heap or balanced structure repeatedly reduces its work. Dynamic-array resizing is an example of **amortized** analysis: individual resizes can take $O(n)$, while repeated doubling yields amortized $O(1)$ pushes.

Review 2 – Stacks: Definition, Operations, and Uses

The LIFO Rule

Push adds a new top item, **pop** removes and returns the top, **peek** reads the top without removing it, and **isEmpty** checks whether an item exists. For a correct stack, the top is always the most recently pushed item not yet popped.



Worked Operation Trace

Operation	Stack bottom → top	Returned result
push(12)	[12]	–
push(25)	[12, 25]	–
push(41)	[12, 25, 41]	–
peek()	[12, 25, 41]	41
pop()	[12, 25]	41
pop()	[12]	25

Implementations and Variants

An **array stack** stores items at positions 0 through $size - 1$; push at $size$, pop at $size - 1$. A **linked stack** adds/removes nodes at the head. A **min stack** tracks the smallest current value, often with a second stack. A **monotonic stack** maintains candidates in increasing or decreasing order; its discarded elements cannot help answer the future question under the chosen rule.

When is a stack suitable?

Choose a stack when the *most recent unfinished action* must be processed first: balancing parentheses, reversing a sequence, navigating nested scopes, undo/redo, or finding a next-greater value. A stack does not provide FIFO service.

Review 3 – Stack Code and Safe APIs

Complete Java Example: Dynamic Array Stack

Save as `StackDemo.java`. This is a reference implementation for understanding index updates and resizing; Problem 1 uses its own command interface.

```
import java.util.Arrays;
import java.util.NoSuchElementException;
public class StackDemo {
    static class IntStack {
        private int[] a = new int[2];
        private int size = 0;
        public boolean isEmpty() { return size == 0; }
        public int size() { return size; }
        public void push(int x) {
            if (size == a.length) a = Arrays.copyOf(a, a.length * 2);
            a[size++] = x;
        }
        public int peek() {
            if (isEmpty()) throw new NoSuchElementException();
            return a[size - 1];
        }
        public int pop() {
            int result = peek();
            size--;
            return result;
        }
    }
    public static void main(String[] args) {
        IntStack s = new IntStack();
        s.push(12); s.push(25); s.push(41);
        System.out.println(s.peek());   // 41
        System.out.println(s.pop());    // 41
        System.out.println(s.size());   // 2
    }
}
```

Java Collection Alternative

For practical application code, Java's `ArrayDeque<E>` supports stack operations `push`, `pop`, and `peek`, as well as queue operations. It does not allow null elements. The older `java.util.Stack` is usually not the preferred starting point for modern new code; use `Deque<E>` where its operations fit the task.

```
import java.util.ArrayDeque;
import java.util.Deque;
Deque<String> undo = new ArrayDeque<>();
undo.push("edit-A"); undo.push("edit-B");
System.out.println(undo.pop());  // edit-B
```

Complexity. Indexed top operations are $O(1)$ except that a growth operation costs $O(n)$; doubling gives amortized $O(1)$ push. A node-based top uses $O(1)$ insert/remove without resizing, but needs an extra reference per node.

Review 4 – Using Stacks to Solve Problems

Nested Brackets

For each opening bracket, push its type. On a closing bracket, require a nonempty stack and a matching opening bracket at the top; then pop. At the end, accept only if the stack is empty. Ignore non-bracket characters only when the problem explicitly permits this.

Character	Stack bottom → top	Action
`{`	[{]	Push
`[`	[{, []]	Push
`(`	[{, [, (]	Push
`)`	[{, []]	Match and pop
`]`	[{]	Match and pop
`}`	[]	Match and pop

Undo and Redo: Two Stacks

Keep one stack of completed actions (undo history) and one of undone actions (redo history). **Undo** moves the latest action from history to redo; **redo** moves it back and reapplies it. A new edit after undo typically clears redo, because the previously undone future is no longer the active history. Specify what one action means before implementing it.

Monotonic Stack: Next Greater Element

For $[30, 32, 31, 35]$, scan left to right and keep indices of positions still waiting for a strictly greater later value. When a larger value appears, resolve and pop eligible indices. Each index is pushed once and popped at most once, giving $O(n)$ total time even though a while loop appears inside the scan.

Review 5 – Queues and Circular Queues

The FIFO Rule

Enqueue inserts at the rear. **Dequeue** removes the oldest item from the front. **Peek** observes the front. A queue is appropriate when arrival order must be respected.



Why a Circular Array?

A fixed ordinary array queue that moves all remaining items after each dequeue can require $O(n)$ time per removal. A **circular queue** retains a `front` index and `size`; physical positions wrap around using modulo K. No shifts are necessary. Capacity K is fixed for this example.

Operation	**front**	**size**	**Logical queue front → rear**
start	0	0	[]
enqueue(14)	0	1	[14]
enqueue(27)	0	2	[14,27]
enqueue(35)	0	3	[14,27,35]
dequeue()	1	2	[27,35]
enqueue(46)	1	3	[27,35,46] (wraps to index 0)

When to Apply

Use circular buffers for fixed-capacity producer/consumer storage or ring buffers. A growing deque is often easier when the maximum capacity is not known. The *circular queue* is array storage with wrapped indices; it is not the same as a *circular linked list*.

Review 6 – Circular Queue Java Code

Complete Example with Wrap-around

Save as `CircularQueueDemo.java`. Empty and full are distinct states.

```
import java.util.NoSuchElementException;
public class CircularQueueDemo {
    static class CircularQueue {
        private final int[] a;
        private int front = 0, size = 0;
        CircularQueue(int capacity) {
            if (capacity < 1) throw new IllegalArgumentException();
            a = new int[capacity];
        }
        int size() { return size; }
        boolean isEmpty() { return size == 0; }
        boolean isFull() { return size == a.length; }
        void enqueue(int x) {
            if (isFull()) throw new IllegalStateException("FULL");
            a[(front + size) % a.length] = x;
            size++;
        }
        int peek() {
            if (isEmpty()) throw new NoSuchElementException();
            return a[front];
        }
        int dequeue() {
            int x = peek();
            front = (front + 1) % a.length;
            size--;
            return x;
        }
    }
    public static void main(String[] args) {
        CircularQueue q = new CircularQueue(3);
        q.enqueue(14); q.enqueue(27); q.enqueue(35);
        System.out.println(q.dequeue());  // 14
        q.enqueue(46);                    // wraps to position 0
        System.out.println(q.peek());     // 27
        System.out.println(q.size());     // 3
    }
}
```

Invariant. The logical ith item is at `a[(front+i) % capacity]` for $0 \leq i < size$. This describes both correctness and wrap-around. All listed operations are $O(1)$ time; the allocated storage uses $O(K)$ space.

Review 7 – Deques and Monotonic Deques

A **double-ended queue (deque)** allows insertions and removals at both ends. In Java, `ArrayDeque<Integer>` provides `addFirst`, `addLast`, `removeFirst`, `removeLast`, and corresponding peeks. Choose it for work needing both ends, not automatically for priority ordering.

From Sliding Sum to Sliding Maximum

In Lab 2, a running sum reused arithmetic between consecutive windows. For a moving **maximum**, subtraction cannot recover the next maximum. A **monotonic deque of indices** stores possible maxima in decreasing value order. Remove expired indices at the front; remove dominated smaller or equal values at the rear; append the new index. The front then points to the maximum in the current complete window.

Worked Trace

For $a = [2, 1, 5, 3, 4, 6, 2]$ and $k = 3$:

Window indices	Window values	Max	Reason
0–2	[2,1,5]	5	5 dominates 2 and 1
1–3	[1,5,3]	5	5 remains in window
2–4	[5,3,4]	5	5 remains at front
3–5	[3,4,6]	6	6 removes smaller candidates
4–6	[4,6,2]	6	6 remains in window

Java Method: Maximum for Each Complete Window

```
import java.util.*;
static int[] windowMax(int[] a, int k) {
    if (k < 1 || k > a.length) throw new IllegalArgumentException();
    int[] answer = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();  // holds indices
    for (int i = 0; i < a.length; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k)
            dq.removeFirst();
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i])
            dq.removeLast();
        dq.addLast(i);
        if (i >= k - 1) answer[i - k + 1] = a[dq.peekFirst()];
    }
    return answer;
}
```

Every index enters once and leaves at most once: $O(n)$ time and $O(k)$ working deque space, plus $O(n-k+1)$ output space. A direct k-element scan per window costs $O((n-k+1)k)$ time.

Review 8 – Priority Queues and Scheduling

Priority Is Different from FIFO

A **priority queue** removes the eligible item with the best priority, not necessarily the oldest item overall. In this lab's examples, smaller priority numbers mean more urgent work; equal priorities use arrival order only when the program defines that secondary rule explicitly.

Complete Java Example

The `PriorityQueue` implementation uses a heap. Items must have a defined comparison; the default Java priority queue is not FIFO for equal keys.

```
import java.util.*;
public class PriorityDemo {
    static class Job {
        int id, priority, arrival;
        Job(int id, int priority, int arrival) {
            this.id=id; this.priority=priority; this.arrival=arrival;
        }
    }
    public static void main(String[] args) {
        PriorityQueue<Job> pq = new PriorityQueue<>(
            Comparator.comparingInt((Job j) -> j.priority)
                      .thenComparingInt(j -> j.arrival));
        pq.add(new Job(101, 3, 0));
        pq.add(new Job(102, 1, 1));
        pq.add(new Job(103, 1, 2));
        while (!pq.isEmpty()) System.out.println(pq.remove().id);
        // 102, 103, 101
    }
}
```

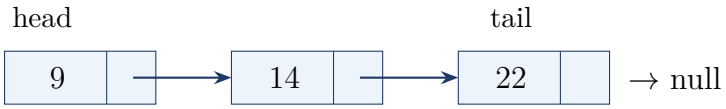
When to Apply

Choose FIFO for ordinary arrival-order tickets, a deque when either end is needed, and a priority queue for urgency-driven dispatch. A binary-heap priority queue generally supports insertion and removal of the priority item in $O(\log n)$, and peeking in $O(1)$. Arbitrary removal of a searched item is not automatically $O(\log n)$, because it may require a linear search.

Review 9 – Singly Linked Lists

Nodes and References

A singly linked node stores `value` and `next`. The list typically keeps `head`; retaining `tail` also makes appending $O(1)$.



Insert and Delete: Preserve the Chain

To prepend, construct a node whose `next` is the old head, then make it the new head. To append with a tail, link the old tail to the new node and update tail. To delete the first occurrence, keep `prev` and `cur`; link around `cur`, updating head or tail as necessary. Java garbage collection reclaims unreachable nodes; there is no manual free operation.

When to Apply

Useful when the sequence changes through frequent known-position link edits, especially at the head. Searching for a value or selecting the ith item still costs $O(n)$ without an external index. A node reference must be treated as a link to a particular node, not as an array index.

Review 10 – Singly Linked List Java Code

Complete Example: Head, Tail, and Removal

```
public class SinglyListDemo {
    static class Node {
        int value; Node next;
        Node(int v) { value = v; }
    }
    static class List {
        Node head, tail;
        int size;
        void addLast(int v) {
            Node n = new Node(v);
            if (tail == null) head = tail = n;
            else { tail.next = n; tail = n; }
            size++;
        }
        void addFirst(int v) {
            Node n = new Node(v);
            n.next = head; head = n;
            if (tail == null) tail = n;
            size++;
        }
        boolean removeFirst(int v) {
            Node prev = null, cur = head;
            while (cur != null && cur.value != v) {
                prev = cur; cur = cur.next;
            }
            if (cur == null) return false;
            if (prev == null) head = cur.next;
            else prev.next = cur.next;
            if (tail == cur) tail = prev;
            size--; return true;
        }
        String values() {
            StringBuilder b = new StringBuilder("[");
            for (Node p = head; p != null; p = p.next) {
                if (p != head) b.append(", ");
                b.append(p.value);
            }
            return b.append("]").toString();
        }
    }
    public static void main(String[] args) {
        List l = new List(); l.addLast(14); l.addLast(22);
        l.addFirst(9); l.removeFirst(14);
        System.out.println(l.values());  // [9, 22]
    }
}
```

Invariant. Empty means `head == tail == null` and `size == 0`. Otherwise tail is reachable from head, `tail.next == null`, and traversing exactly `size` nodes reaches tail. Head insert and tail append take $O(1)$; search and deletion by value take $O(n)$.

Review 11 – Doubly and Circular Linked Lists

Doubly Linked List

Each node stores `prev` and `next`. With head and tail, moving one step forward or backward from a *known node* takes $O(1)$. Removing a known node can take $O(1)$ after repairing both neighbor links; locating a node by value still takes $O(n)$.



Circular Singly and Doubly Lists

A **circular singly linked list** connects the final node back to head instead of null; a circular doubly linked list also connects head's `prev` to tail. These are useful for cyclic scheduling and round-robin work, but traversal must stop on returning to the start or after a known number of steps.

Comparison

Structure	Strength	Caution
Singly	Fewer references per node; fast head operations.	No direct step backward.
Doubly	Easy navigation both ways; delete known node.	More link updates and storage.
Circular singly	Repeated cycle through participants.	No natural null terminator.
Circular doubly	Two-way cyclic navigation.	More invariants to maintain.

Review 12 – Doubly Linked List and Linked Queue

Complete Doubly Linked List Example

```
public class DoublyDemo {
    static class Node {
        int v; Node prev, next;
        Node(int x) { v=x; }
    }
    static class List {
        Node head, tail;
        void addLast(int x) {
            Node p = new Node(x); p.prev = tail;
            if (tail != null) tail.next = p;
            else head = p;
            tail = p;
        }
        void remove(Node p) { // precondition: p is a node of this list
            if (p.prev == null) head = p.next;
            else p.prev.next = p.next;
            if (p.next == null) tail = p.prev;
            else p.next.prev = p.prev;
        }
    }
    public static void main(String[] args) {
        List a = new List(); a.addLast(4); a.addLast(8);
        a.addLast(12); a.remove(a.head.next);
        System.out.println(a.head.next.v); // 12
        System.out.println(a.tail.prev.v); // 4
    }
}
```

A Linked Queue Uses the Same Singly Linked Nodes

Keep `head` (front) and `tail` (rear). Enqueue attaches at tail; dequeue removes head. If dequeue empties the queue, *also set tail to null*. Both operations are $O(1)$ for an already allocated queue. Unlike a fixed circular array, this queue allocates a new node for each arrival. The Java `LinkedList` collection also provides deque operations, but is not a substitute when manual links are being assessed.

How to select a structure

Stack = newest unfinished item; queue = oldest waiting item; deque = both ends; priority queue = priority order; singly list = forward chain; doubly list = forward/backward navigation; circular list = cyclic links. Choose from the required operations, their costs, and storage constraints.

Problem Set at a Glance

This version uses transportation, transit and logistics to practice basic operations, real-world workflows, and interview patterns. Difficulty labels are teaching estimates.

No.	Problem	Focus	Status
1	Cargo Scan Stack	Dynamic array stack	Core
2	Route Instruction Undo and Redo	Two stacks	Choose 2 or 8
3	Shipping Manifest Bracket Check	Stack application	Core
4	Boarding Circular Queue	Fixed array ring buffer	Core
5	Peak Passenger Window	Monotonic deque	Core
6	Next Busier Service Period	Monotonic stack	Choose 6 or 11
7	Editable Stop Registry	Singly linked list	Core
8	Emergency Vehicle Dispatch	Priority queue	Choose 2 or 8
9	Bidirectional Itinerary List	Doubly linked list	Core
10	Station Incident Linked Queue	Linked queue versus ring buffer	Core
11	Cyclic Next-Stop Detection	Floyd's cycle algorithm	Choose 6 or 11
12	Recently Used Station Cache	Map and doubly linked list	Optional

Implementation boundary

Manual data-structure implementation is required only where explicitly named. For a monotonic stack/deque or priority queue application, the stated Java collection may be used. **Using the right library API is not the same task as implementing the data structure's internals.**

Problem 1 – Cargo Scan Stack

Focus: Dynamic array stack; Easy

Scenario and Task

A cargo terminal records freight container IDs during loading. The most recently scanned container can be removed from the active sequence with UNDO. Support the commands `SCAN x`, `UNDO`, `LATEST`, and `COUNT`. An undone scan is removed; this exercise has no redo operation.

Why This Problem Matters

A cargo loader must undo the latest container scan before earlier scans. The active scan history is last-in-first-out, so a stack exposes only the newest still-active container.

Input/Output Contract

Read an integer Q, then exactly Q command lines ($0 \leq Q \leq 10^5$). `SCAN x` adds container x and prints nothing. `UNDO` removes and prints the latest active container or `EMPTY`; `LATEST` prints it without removing or `EMPTY`; `COUNT` prints the number of active scans. container integers may repeat and may be zero or negative.

Suggested Implementation Steps

Keep an array `a` and integer `size`; the active items occupy positions 0 through $size - 1$. When full, double the array capacity before inserting. When popping, read index $size - 1$ before decreasing `size`. Do not use zero as the empty marker because zero can be a legitimate ID.

Required Program

Implement an `IntStack` with a private growing `int[]` and a `size` field. Use doubling when full; provide `push`, `pop`, `peek`, `size`, and `isEmpty`. Implement the command processing yourself. `UNDO` prints the removed ID, `LATEST` prints the current top, `COUNT` prints the number of items; print `EMPTY` when a requested item does not exist. `SCAN` produces no output. Initial array capacity is 2.

Sample Input

```
8
SCAN 45
SCAN 12
LATEST
UNDO
LATEST
COUNT
UNDO
UNDO
```

Expected Output

```
12
12
45
1
45
EMPTY
```

Example Walkthrough

The first two scans create $[45, 12]$ from bottom to top. Both `LATEST` and the first `UNDO` print 12, but only undo removes it. The remaining active scan is 45. After that is removed, a further undo prints `EMPTY`.

Tests and Analysis

Test empty commands, repeated scans across a resize, IDs equal to zero or negative, and clearing then reusing the stack. Trace `size` and physical capacity. Explain the LIFO invariant and distinguish $O(n)$ worst-case resizing from amortized $O(1)$ push. State total storage after n scans.

Problem 2 – Route Instruction Undo and Redo

Focus: Two stacks; Applied choice; Easy–Medium

Scenario and Task

A transport planner writes an itinerary note one token at a time. Undo removes the latest token; redo restores it; any new route edit clears redo history. Treat each `WRITE x` as appending one nonempty token. `UNDO` removes the latest active token, `REDO` restores the latest undone token when possible, and `SHOW` displays the current text.

Why This Problem Matters

A route planner writes instructions in order. One stack stores the current itinerary, and another holds undone instructions for redo; creating a new instruction abandons that redo branch.

Input/Output Contract

Read Q followed by Q command lines. Each `WRITE x` uses a nonempty token without spaces (length at most 30). `WRITE`, valid `UNDO`, and valid `REDO` print nothing. An invalid `UNDO` or `REDO` prints `NOTHING`. `SHOW` prints active tokens in original editing order separated by single spaces, or `EMPTY`.

Suggested Implementation Steps

Treat the top of `history` as the latest written token. An undo pops it onto `redo`; a redo pops from `redo` and pushes it onto `history`. A new write clears `redo`. When displaying the history, show *oldest to newest*, not stack-pop order.

Required Program

Use two Java `ArrayDeque<String>` stacks for the completed and redo histories. Do not use a single array with index arithmetic as a substitute. Allow up to 10^5 commands; token lengths are at most 30 characters.

Sample Input

```
10
WRITE NORTH
WRITE EAST
UNDO
SHOW
REDO
SHOW
UNDO
WRITE DEPOT
REDO
SHOW
```

Expected Output

```
NORTH
NORTH EAST
NOTHING
NORTH DEPOT
```

Example Walkthrough

Undo removes EAST and makes it redoable, leaving just NORTH. Redo temporarily restores EAST. The later undo removes EAST again; writing DEPOT clears the redo history, so the next `REDO` prints `NOTHING` and the final note is NORTH DEPOT.

Tests and Reasoning

Test undo on empty history, redo before any undo, multiple undos, and clearing redo after a new edit. State what each stack represents and why `WRITE` changes the reachable future. Analyze per-operation cost (ignore the cost of printing all current tokens only when analyzing the underlying stack operations).

Problem 3 – Shipping Manifest Bracket Check

Focus: Nested delimiters; Stack; Easy

Scenario and Task

A shipping manifest contains nested parameter delimiters of types (), [], and . Validate correct nesting without treating ordinary text as delimiters. Ignore all characters other than `()`, `[]`, and `{}`. Return one verdict per complete input line.

Why This Problem Matters

A shipping manifest has nested parameter delimiters. Even when opening and closing counts agree, the nesting order can be wrong; a stack matches each closer to its most recent unmatched opener.

Input/Output Contract

Read integer T and then exactly T complete ASCII lines, each possibly empty or containing spaces. For each line print one `VALID` or `INVALID`. Non-bracket characters do not affect the result. $0 \leq T \leq 1000$, maximum line length 10^5. Use line-oriented input rather than token-oriented scanning.

Suggested Implementation Steps

Scan characters from left to right. Push every opening delimiter. For each closing delimiter, reject immediately when the stack is empty or the top has the wrong type; otherwise pop. At end, accept only if no unmatched openers remain. Read the lines with a method that preserves spaces and empty lines (e.g. `BufferedReader.readLine()`).

Required Program

Use a `Deque<Character>` stack. Read characters from left to right, push openings, and ensure every closing bracket matches the most recent opening bracket. Print `VALID` or `INVALID`. A line with no brackets is valid. Assume at most 10^5 characters per line.

Sample Input

```
5
route{stop[1](A)}
([)]
manifest[(3)]
(()
cargo text
```

Expected Output

```
VALID
INVALID
VALID
INVALID
VALID
```

Example Walkthrough

In `([)]`, the closing `)` arrives when `[` is the top unmatched opener, so the line fails despite similar opening/closing counts. The first and third lines close properly nested groups; the fourth has an unmatched opener, and the plain-text fifth line has no brackets.

Tests and Analysis

Test a closing bracket before any opening, unmatched openings remaining at end, and many nested brackets. Explain why checking only bracket counts does not establish correctness. State time and maximum auxiliary space. Do not convert the input into a sorted character sequence.

Problem 4 – Boarding Circular Queue

Focus: Fixed-capacity FIFO, ring buffer; Easy–Medium

Scenario and Task

A bus terminal buffers boarding-ticket IDs in a fixed K-slot ring. Service follows arrival order; newly free slots must be reused without shifting the array. Commands are `ENQ x`, `SERVE`, `FRONT`, and `SIZE`; serving must be FIFO.

Why This Problem Matters

A boarding terminal admits passenger tickets into a bounded waiting buffer. Circular indexing allows the first freed slot to be reused while preserving first-in-first-out service.

Input/Output Contract

Read capacity K and command count Q on the first line, followed by Q commands. `ENQ x` prints `FULL` only when capacity K is reached and otherwise prints nothing. `SERVE` prints and removes the oldest ticket (or `EMPTY`); `FRONT` prints it without removing (or `EMPTY`); `SIZE` prints the queue length. $1 \le K \le 10^5$.

Suggested Implementation Steps

Maintain `front` (the physical index of the oldest boarding ticket) and `size`. Place a new boarding ticket at `(front + size) % K`; on successful service, advance `front` using modulo K and decrease `size`. Decide full/empty using `size`, not slot contents.

Required Program

Implement a **fixed-capacity circular array queue** with `front`, `size`, and modulo K. Do not shift the array after every dequeue. $1 \le K \le 10^5$ and there are at most 10^5 operations.

Sample Input

```
3 8
ENQ 17
ENQ 29
ENQ 33
ENQ 46
SERVE
ENQ 46
FRONT
SIZE
```

Expected Output

```
FULL
17
29
3
```

Example Walkthrough

The three slots first hold 17, 29, and 33; the first enqueue of 46 prints `FULL`. After 17 is served, its physical slot is free. The second enqueue of 46 wraps to that slot. The logical queue is $[29, 33, 46]$, whose front is 29 and size is 3.

Tests and Analysis

Trace physical indices and logical FIFO order through wrap-around and through becoming empty, then reusing the queue. Test $K = 1$. State the invariant `a[(front+i) % K]` and the complexity of each operation. Explain why having an empty slot does not necessarily mean the buffer is logically empty.

Problem 5 – Peak Passenger Window

Focus: Monotonic deque; Core; Medium

Scenario and Task

Passenger counts are recorded in chronological order. Report the peak count in each consecutive complete k-period window, preserving original adjacency. The answer contains one maximum for every complete window, not merely the maximum of the entire series.

Why This Problem Matters

Passenger counts belong to consecutive time periods. When each window advances, the largest value may leave, so a monotonic deque preserves candidate indices rather than trying to subtract an outgoing maximum.

Input/Output Contract

Read n and k, then n passenger counts in chronological order ($1 \leq k \leq n \leq 10^5$). Print exactly $n - k + 1$ maxima separated by one space. Counts are integers; the algorithm should also work for signed test data. The required methods return an `int[]` and must not modify the input.

Suggested Implementation Steps

First implement the direct k-item scan as an answer checker. For the deque method, maintain indices in descending order of their array values. At each new index: (1) remove positions outside the window from the front; (2) remove smaller or equal candidates from the back; (3) append the new index; (4) output the front value once a complete window exists. **Workload limit:** compare both approaches only on small inputs, for example $n \leq 2000$. On larger inputs run the deque algorithm alone.

Required Approaches

A – Direct scan: inspect the k items in every window.

B – Monotonic deque: use `ArrayDeque<Integer>` to store candidate *indices*, evict expired positions from the front, remove dominated candidates from the back, and report the value at the front. Use $1 \leq k \leq n \leq 10^5$. Your method returns an `int[]` of all maxima; the driver prints them on one line, separated by spaces.

Sample Input

```
7 3
9 1 4 7 3 5 2
```

Expected Output

```
9 7 7 7 5
```

Example Walkthrough

The complete length-3 windows are `[9,1,4]`, `[1,4,7]`, `[4,7,3]`, `[7,3,5]`, `[3,5,2]`. Their maxima are 9, 7, 7, 7, 5. The answer includes one maximum per window, not only the overall maximum. The deque must remove an old index once it leaves the current window.

Tests and Analysis

Test $k = 1$, $k = n$, repeated equal maxima, all-negative values, and a previously maximal value expiring. Provide a deque-content trace of the example and show why each index enters and leaves at most once. Analyze $O((n - k + 1)k)$ versus $O(n)$ total time and separate deque storage from output storage. Compare why a running sum alone cannot maintain the maximum.

Problem 6 – Next Busier Service Period

Focus: Monotonic stack; Interview choice; Medium

Scenario and Task

For each passenger count, find how many later periods pass before the next strictly greater count. Equal-count days do not qualify. If no later value is strictly greater, return zero; do not count equal readings as an improvement.

Why This Problem Matters

For each service period, planners need the distance to the next strictly busier period. A monotonic stack holds earlier periods awaiting a higher future count and resolves them when it arrives.

Input/Output Contract

Read n ($0 \leq n \leq 10^5$) then n integer passenger counts. Print n waiting distances, separated by one space, or a blank line when $n = 0$. For period i, if period $j > i$ is the first later period with $a[j] > a[i]$, output $j - i$; otherwise output 0.

Suggested Implementation Steps

Initialize an output array with zeros. Scan left to right. While the stack is nonempty and the new reading exceeds the reading at its top index, pop that earlier index j and set the answer at j to $i - j$. Finally push i. Do not pop on equality. The stack contains unresolved indices.

Required Program

Use a `Deque<Integer>` stack of unresolved *indices*. As each new value is read, pop earlier indices while their values are strictly smaller, recording the distance for each. Compare with a direct $O(n^2)$ approach in your report, but only the stack implementation is required.

Sample Input

```
5
6 9 8 10 9
```

Expected Output

```
1 2 1 0 0
```

Example Walkthrough

At index 0 (6), the next strictly larger value is 9, one position later. At index 1 (9), the first strictly higher value is 10 at index 3, so the distance is 2. Index 2 resolves at index 3; the final two indices have no strictly larger later value. Equality would not qualify.

Tests and Analysis

Test all-equal, strictly increasing, strictly decreasing, and negative values. Explain the invariant governing unresolved indices and why popping an index is safe. State $O(n)$ time and $O(n)$ extra space; distinguish one pop per index from the apparent nested loops.

Problem 7 – Editable Stop Registry

Focus: Custom singly linked list; Easy–Medium

Scenario and Task

A transit planner maintains an editable forward linked chain of stop IDs. Support insertion at both ends, first-match removal, lookup, and display. Implement `ADD_FIRST x`, `ADD_LAST x`, `REMOVE x`, `FIND x`, and `LIST`.

Why This Problem Matters

A stop registry supports urgent additions at the front and regular additions at the rear without shifting every stop in an array. A linked list changes references directly, though finding a stop by ID still requires traversal.

Input/Output Contract

Read Q followed by Q commands. Both addition commands print nothing. `REMOVE x` deletes only the first node with value x and prints `REMOVED` or `NOT_FOUND`. `FIND x` prints the zero-based index of the first match or `-1`. `LIST` prints the current IDs from head to tail, space-separated, or `EMPTY`. Duplicate IDs are allowed.

Suggested Implementation Steps

Maintain `head`, `tail`, and `size`. When adding the first node, both ends must point to it. To remove a value, track `previous` and `current`; bypass the first matching node and repair `head` or `tail` if necessary. If the last node is removed, both head and tail must become null.

Required Program

Implement your own `Node {int value; Node next;}` and a list with `head`, `tail`, and `size`. Do not use Java's built-in `LinkedList`. Maintain tail correctly when deleting the final node.

Sample Input

```
6
ADD_LAST 36
ADD_LAST 48
ADD_FIRST 7
REMOVE 36
FIND 48
LIST
```

Expected Output

```
REMOVED
1
7 48
```

Example Walkthrough

After three additions the head-to-tail sequence is $7 \to 36 \to 48$. Deleting 36 bypasses that node; the first occurrence of 48 is now at zero-based index 1. The final list is `7 48`.

Tests and Analysis

Include empty removal, deleting head, deleting tail, deleting the only node, and duplicates. Explain the difference between removing a *known node* and finding a node by value. Give time costs for head insertion, tail append, find, remove-by-value, and traversing all entries.

Problem 8 – Emergency Vehicle Dispatch

Focus: Priority queue; Applied choice; Easy–Medium

Scenario and Task

Road incidents compete for emergency vehicles. Smaller numeric priorities mean higher urgency; on ties process the earliest report. Process `ADD id priority` and `SERVE`. Identical IDs are allowed and arrival order is a separate property.

Why This Problem Matters

Emergency vehicle requests should be served by urgency, not simply arrival order. A priority queue chooses the most urgent outstanding incident and preserves arrival order only for equal priorities.

Input/Output Contract

Read integer Q followed by exactly Q commands. `ADD id priority` creates a new incident and prints nothing. `SERVE` removes and prints the waiting incident with smallest priority number, breaking ties by earlier insertion; on empty input it prints `EMPTY`. IDs can repeat, so do not use incident ID as the arrival tiebreaker.

Suggested Implementation Steps

Create a `Job` containing ID, priority, and monotonically increasing arrival number. Compare priority first, arrival second. Do not use the ID as a tiebreaker. The priority and arrival fields do not change after insertion.

Required Program

Use `PriorityQueue<Job>` with a comparator on `priority`, then `arrival`. Write the Job type, comparator, and driver. Do not assume Java's queue is automatically stable for equal keys. Job IDs may be reused; arrival number is separate and strictly increasing.

Sample Input

```
7
ADD 801 3
ADD 802 1
ADD 803 1
SERVE
SERVE
SERVE
SERVE
```

Expected Output

```
802
803
801
EMPTY
```

Example Walkthrough

Although request 801 arrived first, 802 and 803 have higher urgency (priority 1 rather than 3). Equal-priority requests are served in arrival order: 802, then 803. Request 801 follows and the last `SERVE` prints `EMPTY`.

Tests and Reasoning

Test equal priorities, different priorities, empty removal, and multiple interleaved additions/removals. Compare the result with ordinary FIFO; explain why the two policies differ. State heap insertion/removal complexity and peek complexity, noting that priorities are fixed after insertion.

Problem 9 – Bidirectional Itinerary List

Focus: Custom doubly linked list; Core; Medium

Scenario and Task

A transit itinerary needs forward and reverse navigation through its ordered stop IDs, even after inserts and removals. Implement `ADD_FIRST x`, `ADD_LAST x`, `REMOVE x`, `FORWARD`, and `BACKWARD`.

Why This Problem Matters

A transit itinerary must remain readable forward and backward after editing stop IDs. A doubly linked list maintains both next and previous links and requires both directions to be repaired on removal.

Input/Output Contract

Read Q and Q commands. Addition commands print nothing. `REMOVE x` deletes the first matching node and prints `REMOVED` or `NOT_FOUND`. `FORWARD` displays head-to-tail stop IDs and `BACKWARD` displays tail-to-head IDs, each separated by spaces; either prints `EMPTY` when there are no stops. Repeated stop IDs are allowed.

Suggested Implementation Steps

Each node contains both `prev` and `next`. Maintain head, tail, and size. For removal, search from head for the first equal value; link the two neighbors around it and adjust whichever end it occupied. Afterward verify both forward and backward traversals. Locating a node by value is still $O(n)$; only unlinking an already known node is $O(1)$.

Required Program

Implement `Node {int value; Node prev, next;}` with head, tail, and size; repair both directions on each change. Do not use `java.util.LinkedList`.

Sample Input

```
7
ADD_LAST 36
ADD_LAST 48
ADD_LAST 59
REMOVE 48
ADD_FIRST 7
FORWARD
BACKWARD
```

Expected Output

```
REMOVED
7 36 59
59 36 7
```

Example Walkthrough

The initial chain is $36 \leftrightarrow 48 \leftrightarrow 59$. Removing 48 must connect 36 to 59 in both directions. Adding 7 at the head yields forward order 7, 36, 59 and backward order 59, 36, 7.

Tests and Analysis

Test an empty list, singleton, deletion at both ends, duplicate keys, and alternating head/tail additions. Explain why an incorrect `prev` link can pass a forward-traversal test but fail a backward traversal. Compare the extra references per node and deletion complexity with a singly list; note that finding a value remains linear.

Problem 10 – Station Incident Linked Queue

Focus: Custom singly linked queue; Medium

Scenario and Task

A station office serves incoming incident IDs in arrival order. An unbounded linked queue must maintain both front and rear references. Support `ENQ x`, `SERVE`, `FRONT`, and `SIZE`. There is no fixed capacity or FULL response.

Why This Problem Matters

A station office serves incident requests in arrival order but does not want a fixed-size buffer. A linked queue keeps references to both ends, allowing new arrivals and service without scanning the whole chain.

Input/Output Contract

Read integer Q followed by Q commands. `ENQ x` inserts at the rear without output. `SERVE` removes and prints the oldest incident, or `EMPTY`; `FRONT` prints but does not remove it, or `EMPTY`; `SIZE` prints the number of queued incidents. There is no `FULL` result in this problem.

Suggested Implementation Steps

Use singly linked nodes and maintain head, tail, and size. For enqueue, attach to tail; for dequeue, detach head. If the removed node was the only one, set `tail = null` too. Do not traverse to the end on every insertion. Contrast physical storage with Problem 4 while preserving the same observable FIFO rule.

Required Program

Implement the queue using singly linked nodes, `head`, `tail`, and `size`. All enqueue, serve, front, and size operations must be $O(1)$. Do not traverse the list to find its tail on each enqueue. Compare it with the fixed circular-array implementation in Problem 4.

Sample Input

```
9
ENQ 19
ENQ 26
SERVE
ENQ 38
FRONT
SIZE
SERVE
SERVE
SERVE
```

Expected Output

```
19
26
2
26
38
EMPTY
```

Example Walkthrough

The first service removes 19 and leaves 26. Enqueuing 38 yields front-to-rear order $[26, 38]$. The next two serves empty the queue. The final serve must print `EMPTY`, not follow a stale tail reference.

Tests and Analysis

Test the queue becoming empty and being reused, a single item, empty removal, and 1,000 alternating operations. Show why removing the last item must reset `tail` to null. Compare capacity management, memory, and the work per operation against circular arrays. Do not claim every linked structure is faster than every array representation.

Problem 11 – Cyclic Next-Stop Detection

Focus: Floyd's two-pointer algorithm; Interview choice; Medium

Scenario and Task

A transit-stop successor chain may link its tail to an earlier Node, making route-following never terminate. Detect this structural cycle. Equal values stored in different nodes are not evidence of a cycle.

Why This Problem Matters

A next-stop reference that points back to an earlier stop creates an infinite route-following loop. Floyd's slow/fast pointers detect repeated node identity without relying on stored stop IDs.

Input/Output Contract

Read n and `pos` on the first line, then n integer stop IDs ($0 \leq n \leq 10^5$). The input builds a list in given order. If `pos=-1`, the final node's `next` is null; otherwise $0 \leq pos < n$ makes the final node link to the zero-based node at position `pos`. Print `true` for a cycle, otherwise `false`; require `pos=-1` when $n = 0$.

Suggested Implementation Steps

Create actual `Node` objects, connecting the last to position `pos` only when requested. Start `slow` and `fast` at head. Repeatedly move slow one edge and fast two edges while both steps are possible. Equal *references* mean a cycle; reaching null means no cycle. Never compare node data values to decide cycle identity.

Required Program

Implement Floyd's slow/fast pointer method on actual singly linked `Node` objects. Build test lists using a helper that connects the final node to zero-based position `pos`, or leaves the last link null when `pos=-1`. The algorithm must terminate even when a cycle exists.

Sample Input

```
4 1
13 26 39 52
```

Expected Output

```
true
```

Example Walkthrough

Position 1 contains 26. The final node (52) links back to that *node*, creating 26, 39, 52, 26, ... in the successor chain. With `pos=-1`, the chain would end at null. Equal values in separate nodes do not themselves create a cycle.

Tests and Analysis

Test empty, one node with and without self-loop, link to head, link to middle, and repeated equal values with no cycle. Explain why `slow == fast` checks *node identity*, not node value. Give $O(n)$ time and $O(1)$ auxiliary space and compare conceptually with a HashSet of visited nodes. Do not print a potentially cyclic chain using an unbounded loop.

Problem 12 – Recently Used Station Cache

Focus: Optional challenge; Map + doubly linked list

Scenario and Task

A route planner keeps a fixed number of recently accessed station-data entries; refreshing a key moves it to the most-recent position and overflow evicts the least recent. Implement `GET key` and `PUT key value`, with most-recent access at the head of the internal recency list.

Why This Problem Matters

A route-planning cache should retain recently accessed station records and evict the least recently used when full. The map locates entries; a doubly linked list makes recency moves inexpensive.

Input/Output Contract

Read positive capacity K, operation count Q, then Q command lines. `PUT key value` produces no output. `GET key` prints the cached value or `-1` if missing. Values may be signed integers, including -1; use map membership internally to distinguish a stored -1 from a missing key. Capacity limits distinct stored keys, not total operations.

Suggested Implementation Steps

Keep most-recently used nodes at the head. Every successful get and every put of an existing key moves its node to the head. On a new put when full, remove the tail key from both the list and map before inserting the new key. Verify that the list contains exactly one node per key in the map.

Required Design

Combine a `HashMap<Integer, Node>` with a manually linked doubly list. Map a key directly to its node; keep most-recent near the head and least-recent near the tail. Demonstrate $O(1)$ *expected* time per operation under ordinary hashing assumptions. Do not scan the list for each GET.

Sample Input

```
2 6
PUT 4 40
PUT 8 80
GET 4
PUT 12 120
GET 8
GET 12
```

Expected Output

```
40
-1
120
```

Example Walkthrough

The initial keys are 4 and 8. Getting 4 makes it most recently used, so inserting 12 at capacity two evicts 8. The subsequent get for 8 returns -1, while 12 remains cached with value 120.

Analysis

Explain why moving an accessed node requires a *doubly* linked removal, how eviction updates the map, and why the node count must never exceed K. Test updating an existing key and capacity 1. This is optional and not a replacement for the seven core tasks or the two selected choices.

Interview Corner – Reasoning and Debugging

Complete **Interview Corner A–D in the PDF report only**. Reasoning, short traces, pseudocode, or corrected snippets are sufficient. **No additional runnable Java programs are required for A–D.** This differs from the selected programming Interview Choice (Problem 6 or 11).

Interview A – FIFO Versus LIFO

A station desk records arrivals $10, 20, 30$. Compare the output when the next two items are removed from a queue versus a stack. Explain which policy fits a customer waiting line and why calling the same `pop` method name would not make two structures behaviorally equivalent.

Interview B – Broken Tail Pointer

In a singly linked queue, a programmer removes the final node using `head = head.next` but does not reset `tail`. Describe the state of head and tail after removal, what can happen when a new item is enqueued, and the correction. State the empty-queue invariant.

Interview C – Why a Circular Queue Wraps

For capacity 3, the physical array initially holds $[7, 8, 9]$ with `front=0,size=3`. One `SERVE` is followed by `ENQ 10`. Write the physical array, new front and size, and logical queue order. Explain why a naive append-at-index-3 rule is wrong.

Interview D – Which Data Structure Fits?

For each situation, choose a data structure or variant and justify the key operation and its complexity: (1) recent-action undo; (2) strict arrival-order check-in; (3) urgency-based priority-based dispatch; (4) next greater sensor reading; (5) frequent two-way playlist navigation; (6) round-robin participant sequence; (7) reporting every rolling-window maximum; and (8) recognizing a cycle without remembering every visited node.

Technique Recognition and Comparison

Recognition Table (Report Only)

For each case state *structure/variant, essential property, and expected operation cost.* Distinguish built-in collections from manually implemented list or queue internals.

Case	Situation
A	Push and undo the latest container scan in a command history.
B	Process requests fairly using a bounded boarding buffer whose slots wrap.
C	Keep candidates for maxima of every length-k window.
D	Remove the most urgent waiting task, with FIFO ties.
E	Insert at tail and remove at head without shifting an array.
F	Navigate backward from a known checkpoint node.
G	Serve cyclic participants using node references.
H	Detect whether next links ever revisit a node.

Algorithm and Storage Comparison Table (Report Only)

Structure	Insertion / removal	Search / navigation	Storage or precondition
Array stack			
Circular array queue			
Priority queue (heap)			
Singly linked list			
Doubly linked list			
Circular linked list			
Monotonic deque			

Reflection

Explain one case where a linked list is preferable to shifting an array, and another where an array is preferable to a linked list. What invariant was hardest to maintain? Which failing or boundary test exposed the error?

Report Requirements

Submit one PDF with your full name, student ID, course, **Lab 3 – Version 4**, the completed problem numbers, and the two choice selections. Do not paste entire source files into the report.

For Each Completed Programming Problem

1. Describe the selected ADT/representation and the key algorithm.
2. Provide a brief invariant or correctness explanation; include a small trace when a problem requests one.
3. State time and auxiliary-space complexity for each required approach, including amortized or expected assumptions when relevant.
4. Answer its analysis questions, including representation trade-offs.
5. Provide normal, boundary, and tricky test inputs with expected and actual output.

Test Record Template

Case	Input	Expected	Actual
Normal			
Boundary			
Tricky			

Comparison Table

Use this where two required approaches or representations are compared.

Criterion	Approach A	Approach B
Main invariant and idea		
Time complexity		
Auxiliary / stored-node space		
Main advantage		
Main limitation		
Suitable workload		

Written work. Include Interview Corner A–D, the technique-recognition table, the completed comparison table, and reflections in this PDF; **do not create separate Java programs for the written questions**. Optional problems should be identified explicitly as optional.

Submission Requirements and Checklist

Individual Submission Package

Submit one compressed ZIP containing actual Java files, one PDF report, and a `README.txt` showing the Java version and exact compile/run commands. Use only the selected choice folders. The example structure is:

```
StudentID_FullName_Lab03_V4.zip
  src/
    Problem01_ScanStack/
    Problem03_Brackets/
    Problem04_CircularQueue/
    Problem05_WindowMaximum/
    Problem07_SinglyTickets/
    Problem09_DoublyCatalogue/
    Problem10_LinkedQueue/
    Problem02_UndoRedo/       (choose 02 or 08)
    Problem08_PriorityJobs/   (choose 02 or 08)
    Problem06_NextWarmer/     (choose 06 or 11)
    Problem11_CycleCheck/     (choose 06 or 11)
    Problem12_LRU/            (optional)
    common/                   (if needed)
  report/
    Lab03_V4_Report.pdf
  README.txt
```

Compiled `.class` files and IDE build folders are unnecessary. Include all custom helper classes used by submitted solutions; screen captures do not replace source code.

Final Checklist

- Seven core solutions and one chosen solution from each choice group compile and run.
- Manual implementations are used where requested; built-in collections are used only when allowed by the problem.
- Empty/singleton cases, head/tail updates, circular wrap-around, and ties are tested.
- Analysis distinguishes logical behavior, physical representation, and work complexity.
- Required examples, traces, and at least three tests per completed program are reported.
- The PDF includes Interview Corner A–D and the written comparison exercises.

Final objective

A strong solution chooses the *appropriate data structure*, states the invariant it must preserve, handles boundary cases, and explains how the implementation affects cost. The goal is not merely to memorize an API.